PROJECT ARCHITECTURE BLUEPRINT: EXOPLANETARY CHARACTERIZATION & ASTROCHEMISTRY SUITE (ECAS)

---

1. EXECUTIVE VISION & OVERALL GOAL

The ultimate objective of this project is to build a unified, standalone, publication-grade desktop application for **Exoplanetary Characterization and Astrochemistry (ECAS)**. The software bridges the gap between quantum-mechanical molecular modeling and observational astrophysics.

By inputting observational data—specifically stellar transit light curves and atmospheric transmission spectra—the suite will allow users to determine an exoplanet's physical dimensions, orbital parameters, atmospheric chemical composition, temperature, and pressure profiles.

Core Workflows

- **The Forward Engine:** Runs high-throughput quantum chemistry calculations to determine the exact infrared vibrational transitions of gaseous molecules from first principles, converting these quantum outputs into macroscopic atmospheric transmittance windows using thermodynamic line-broadening models.
- **The Inverse Engine:** Ingests noisy, raw observational data (such as JWST or Hubble transmission spectra) and applies non-linear deconvolution algorithms to reverse-engineer the precise chemical abundance profiles, identifying known biosignatures/greenhouse gases while systematically flagging and identifying unknown chemical anomalies.
- **The Observational & Visual Engine:** Extracts planetary characteristics directly from stellar photometry (light curves) and mirrors the entire system inside a hardware-accelerated 3D environment, where orbital transits are synchronized dynamically with data charting grids.

---

2. CURRENT DEVELOPMENT STATUS ("WHAT WE HAVE THUS FAR")

The foundational mathematics and the data pipeline architecture have been completely validated and prototyped within a unified Wolfram Mathematica environment. The core prototype includes:

Verified Computational Infrastructure

First-Principles Peak Harvesting

A bulletproof regular expression (Regex) string parsing engine has been engineered to scan raw text output files (.out) generated by the **ORCA Quantum Chemistry Software Suite** (calculated using the r2SCAN-3c functional density model for geometry optimization and

frequency calculations). The parser strips away file formatting variations, targets the IR SPECTRUM block, isolates the coordinate boundaries, and extracts fundamental vibrational modes (wavenumber in cm^{-1}) alongside their corresponding transitions (dipole intensity in km/mol).

Vectorized Line-Broadening Math

An algebraic modeling script handles atmospheric line shape modeling. Rather than running slow execution loops, the code maps harvested peak matrices across a normalized tracking variable vector ($t \in [0, 1]$) utilizing highly optimized array transformations:
$$\text{Lorentzian}[x, x_0, \gamma] = \frac{1}{\pi} \left(\frac{\gamma}{(x - x_0)^2 + \gamma^2} \right)$$
$$\text{Optical Depth } (\tau) = \text{Concentration} \times \sum (I_i \times \text{Lorentzian}[x, \nu_i, \gamma])$$

Real-Time Dual-Unit Visualization Dashboard

A fully interactive dashboard interface has been completed. It models a complex **5-Gas Atmosphere Mixture** consisting of:

1. **Water Vapor (H_2O)**
2. **Carbon Dioxide (CO_2)**
3. **Methane (CH_4)**
4. **Ozone (O_3)**
5. **Nitrous Oxide (N_2O)**

The dashboard features real-time slider controls for line-broadening width (γ) and individual gas layer multipliers. It implements a true exponential **Beer-Lambert Law** formulation:

$$\text{Transmittance} = e^{-\tau_{\text{total}}}$$

When concentrations scale up, peaks realistically flatten and saturate at exactly 0.0 transmittance.

Multi-Axis Transformation Grid

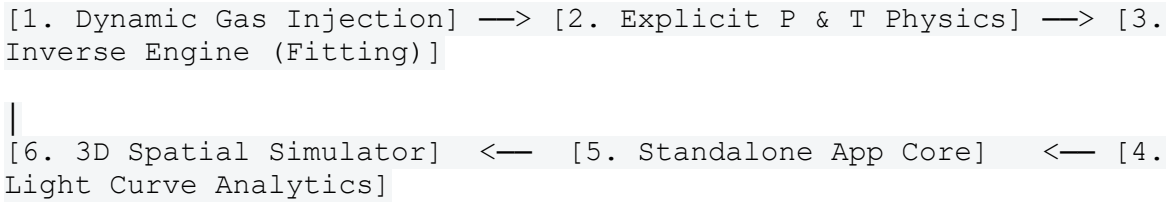
The engine includes a native unit conversion layer that seamlessly warps both the physical equations and the graph limits between laboratory chemical scales (**Wavenumbers in cm^{-1}, spanning 400 to 4200**) and satellite observational scales (**Wavelengths in Micrometers, μm, spanning an ultra-wide viewport of 0.1 to 25.0**). The conversion follows the inverse relationship:
$$\lambda = \frac{10000}{\nu}$$

The graphing grid remains structurally stable, with all negative coordinate mapping space programmatically removed.

---

3. COMPREHENSIVE ROADMAP ("WHAT WE WANT TO DO")

To transition this prototype into a standalone application, six advanced phases of development must be executed:



1. Dynamic Gas Injection & Portfolio Scaling

- Eliminate hardcoded gas variables entirely.
- Implement a dynamic configuration manager where adding a gas is as simple as registering an entry in a JSON object pointing to its ORCA .out file track.
- The interface must parse the file on startup, generate a dedicated slider with the correct gas name, allocate its color code, and mix its optical depth profile into the master summation function natively.

2. Explicit Temperature & Pressure Physics (HITRAN Standard)

- Replace the arbitrary "line broadening" control slider with two explicit physical inputs: **Atmospheric Pressure (P in atm or bar)** and **Atmospheric Temperature (T in Kelvin)**.
- **Pressure Broadening (Lorentzian HWHM):** Implement collisional widening where the width scales linearly with pressure:
  $$\gamma_L(P, T) = \gamma_0 \left(\frac{P}{P_0}\right) \left(\frac{T_0}{T}\right)^n$$
- **Doppler Broadening (Gaussian HWHM):** Implement temperature-driven velocity widening based on molecular mass (M):
  $$\alpha_D(T) = \nu_0 \sqrt{\frac{2 k_B T \ln 2}{m c^2}}$$
- **Unified Profile:** Combine both into a true **Voigt profile** approximation to simulate gas behavior under extreme conditions, from hot exoplanets down to freezing planetary atmospheres.

3. The Inverse Fitting Engine & Anomaly Finder

- Build a data ingestion layer that imports CSV/DAT files from observational archives (e.g., JWST transit spectra).
- Implement a non-linear optimization routine (**Levenberg-Marquardt Least Squares Regression** or **Markov Chain Monte Carlo - MCMC** sampling). The engine will automatically wiggle the gas concentration sliders until the simulated forward transmittance model achieves a minimal chi-squared (χ^2) error matching the real data curve.
- **Residual Analysis & Anomaly Flagging:** Generate an automated "Residual Grid" by calculating:
  $$\text{Residual}(x) = \text{Data}_{\text{Observed}}(x) - \text{Model}_{\text{Best-Fit}}(x)$$
- If a residual peak spikes past a user-defined noise threshold (e.g., 3σ), the app flags it as an **"Unidentified Chemical Anomaly"**.
- **Reverse Identification Routine:** The engine sweeps the peak locations of these anomalies and queries a local look-up table of spectral profiles to suggest candidate

molecules (e.g., identifying volcanism via SO_2 or industrial pollutants like chlorofluorocarbons).

4. Transit Photometry (Light Curve) Analytics

- Ingest stellar flux time-series data (light curves).
- Implement automated curve analysis to extract orbital geometries:
    - **Transit Depth (ΔF):** Calculate the exact physical size of the planet relative to the star using:
      $$\frac{\Delta F}{F} \approx \left(\frac{R_p}{R_*}\right)^2$$
    - **Transit Duration & Period (P):** Deduce the planet's orbital speed, semi-major axis (a), and estimate its planetary equilibrium temperature based on stellar flux distances.
    - **Transmission Spectroscopy Extraction:** Calculate transit depths across separate wavelengths. Because an atmosphere blocks extra starlight only at specific color frequencies, parsing the tiny depth changes across wavelengths maps out the planet's transmission spectrum directly from raw starlight dips.

5. Standalone Software Architecture & Project Manager

- Package the code into a multi-threaded, hardware-accelerated desktop application.
- Implement an intuitive **Molecule Builder UI** allowing users to either search a database or construct a chemical structure via an elements brush. If the molecule's data is uncalculated, the software generates a formatted .inp deck, calls an internal ORCA process silently in a background thread, caches the output matrix locally, and populates a new interface control slider.
- Build a state-saving project manager supporting full .json file saves of dashboard arrays, active gas mixes, and imported datasets alongside high-resolution graphic exports (.svg, .pdf, .png) containing structured source citations.

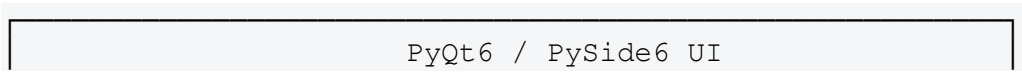
6. Interactive 3D Spatial Environment Simulator

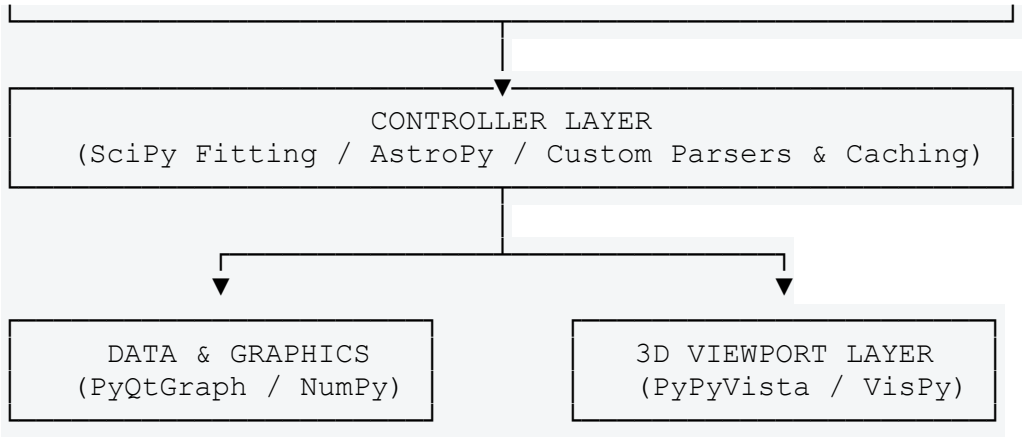
- Build a 3D visualization panel using a graphics engine to render the target host star and its orbiting exoplanetary system using true, calculated scale metrics.
- **Synchronized Data Mirroring:** Wire the 3D space animation timeline directly to the active graphing screens. As the planet physically sweeps across the stellar disk in the 3D viewport, a synchronized tracker bar must scrub across the active light curve and transmittance spectrum charts simultaneously.

---

4. METHODOLOGY & TECH STACK ("HOW WE CAN DO IT")

To maximize processing efficiency, scalability, and layout responsiveness, the application will be written using a modern **Python-driven scientific stack**.





Frontend UI & Layout Framework

- **PyQt6 / PySide6:** Provides a responsive, professional framework for building multi-panel window grids, collapsible slider docks, tabbed plot viewports, and custom dark-theme styling layouts.

Core Calculation Engine

- **NumPy & Cndarray/Numba:** All Lorentzian/Gaussian/Voigt lineshape calculations must be vectorized inside NumPy memory blocks. Utilizing Numba's @njit decorator will compile these loops into machine code, guaranteeing instant, lag-free graphics rendering during real-time slider updates.
- **SciPy (scipy.optimize):** Leverages the Levenberg-Marquardt optimizer (least_squares) and curve-fitting algorithms for high-speed spectral deconvolution.

Observational Astrophysics Calculations

- **AstroPy & Lightkurve:** Handles standard coordinate systems and atmospheric attenuation corrections, while reading and detrending raw stellar transit datasets natively.

High-Performance Graphing Panels

- **PyQtGraph:** Avoid standard Matplotlib pipelines, as they introduce severe rendering lag under fast real-time slider manipulation. PyQtGraph maps arrays straight to OpenGL routines, allowing the app to comfortably re-chart thousands of datapoints at 60+ FPS while sliders are actively dragged.

3D Hardware-Accelerated Viewport

- **PyVista or VisPy:** Wraps raw OpenGL shader code to render smooth geometric spheres, ambient lighting maps, stellar flares, and planetary transit animations directly on the user's graphics card (GPU).

---

5. REPOSITORIES, DATABASES, & ACADEMIC SOURCES

The system relies on external databases and literature parameters to ground its forward and inverse models in real-world data:

Core Databases to Query/Cache

- **The NASA Exoplanet Archive (TAP API):** Used to query confirmed orbital timelines, stellar radii (R_*), distances, and light curve telemetry data points programmatically.
- **The HITRAN / HITEMP Databases (HITRANonline API):** The gold standard for verified laboratory molecular physics parameters. Provides accurate transition line centers (ν_0), air-broadening coefficients (γ_{air}), and temperature exponent parameters (n) for checking first-principles ORCA outputs.
- **PubChem PUG REST API:** Used by the Molecule Builder interface to resolve chemical name string tokens into clean 3D Cartesian Coordinate grids (XYZ in Angstroms) to generate automated ORCA input scripts.
- **ExoMast (MAST API):** Provides access to space telescope datasets from the Hubble, Kepler, TESS, and JWST archives.

Critical Academic References (To Embed in Citation Engine)

- *Rothman, L. S., et al. "The HITRAN 2012 molecular spectroscopic database." Journal of Quantitative Spectroscopy and Radiative Transfer.* (Essential for atmospheric cross-sections).
- *Seager, S. "Exoplanet Atmospheres: Physical Processes." Princeton University Press.* (Establishes the foundation for transit depth extraction mathematics).
- *Neese, F. "The ORCA program system." Wiley Interdisciplinary Reviews: Computational Molecular Science.* (Documents the quantum engine functional background calculations).

---

6. PRODUCTION IMPLEMENTATION GUIDE (STEP-BY-STEP DEV PATH)

Follow this structured development sequence to construct the ECAS standalone software architecture sequentially:

Phase 1: Environment Assembly & Core Architecture

1. Initialize a localized Python repository using an enterprise virtual environment system (venv or conda).
2. Install primary dependencies: pip install PyQt6 pyqtgraph numpy scipy astropy lightkurve pyvista.
3. Set up a rigid file folder directory:
    - /app/ui/ (Layout templates and slider structures)
    - /app/core/ (Physics functions, solvers, and regex parsing modules)
    - /app/data/ (Cached chemical peak dictionaries and structural JSON sheets)
    - /output/ (Vector rendering exports and calculated logs)

Phase 2: Building the Vectorized Physics Pipeline

1. Write the dynamic regex parser inside core/parser.py. It should safely isolate the

boundaries from "IR SPECTRUM" to the line of asterisks "\n*", harvest column values, and map them to a clean nested numpy dictionary array.
2. Code the thermodynamic Voigt or Lorentzian lineshape functions within core/physics.py. Utilize vectorized NumPy syntax (np.sum across multidimensional broadcasted axes) to eliminate hidden loop lag entirely.
3. Establish a standard configuration manager class (core/config.py) that loads a local JSON file listing active molecules and their respective ORCA output directory tracks.

Phase 3: Coding the PyQtGraph Real-Time Canvas

1. Design the main window grid (ui/main_window.py) featuring a vertical slider layout pane on the left and a prominent PyQtGraph canvas viewport filling the right panel.
2. Initialize two discrete unit tracking tabs on the graphing viewport: Wavenumber and Wavelength. Lock the X-axis limits securely to a static tracking index vector ($t \in [0, 1]$), mapping the underlying physical points down to either a hardbound micrometer layout ($0.1 \rightarrow 25.0\,\mu\text{m}$) or a wavenumber layout ($400 \rightarrow 4200\,\text{cm}^{-1}$).
3. Connect the PyQt6 slider signals to the main data update thread. When a slider moves, recalculate the master transmission vector using the pre-cached memory matrices, passing the updated values directly into the plot line item using the .setData() command.

Phase 4: Constructing the Inverse Fitting & Anomaly Flagging Framework

1. Build an import system allowing users to drag and drop external CSV spectral data files into the canvas layout.
2. Wire a button that invokes the scipy.optimize.least_squares routine. Map the parameters so the solver treats the slider values as active boundaries, automatically adjusting them to match the loaded CSV file profile.
3. Write the anomaly tracking handler. Compute the absolute difference between the optimized forward simulation and the loaded observational line. If any remaining peak rises past a 3σ noise boundary, record its precise coordinate index, highlight it on the graph using red warning bars, and display an alert list labeled: "Unidentified Molecular Signature Detected".

Phase 5: Integrating Transit Analytics & 3D Synchronization

1. Code an extraction module using lightkurve to process stellar photometry streams. Implement the math routines to extract planetary radius metrics (R_p) and orbital velocities from the transit depths.
2. Build the 3D visualization panel using PyVista, embedding it straight into a tabbed layout block inside the user interface window. Map a moving sphere mesh to follow a rigid Keplerian orbit profile around a central point light source.
3. Connect the 3D planetary clock directly to the graphing screens. As the user scrubs a time slider, update the position of the planet in the 3D window while simultaneously moving a vertical tracker line across the light curve plot panel.

Phase 6: Packaging, Caching, & Deployment

1. Implement a complete project configuration serialization framework via Python's json

module, enabling users to freeze their entire environment layout state, active chemical portfolios, and loaded data views into portable file tokens.
2. Implement vector output export hooks utilizing PyQtGraph's built-in SVGExporter and CSVExporter parameters to cleanly route high-fidelity vector graphics and raw processed numbers directly into the /output/ folder structure.
3. Freeze the final workspace environment tree into a standalone desktop executable file payload across your target platform deploying using PyInstaller:

```
pyinstaller --noconfirm --onedir --windowed --add-data
"app/data;data" main.py
```

---

7. PROMPT FOR TARGET AI DEVELOPER (COPY-PASTE READY)

```
You are an expert software architect and computational physicist
specializing in Python, PyQt6, and vectorized scientific computing.
Your task is to implement the Exoplanetary Characterization &
Astrochemistry Suite (ECAS) based on the comprehensive blueprint
layout provided above.

Key Implementation Directives:
1. Performance First: Avoid Matplotlib. Use pyqtgraph for all 2D
charts to ensure real-time rendering performance (60+ FPS) when
users interact with the sliders.
2. Vectorization: Use NumPy array broadcasting and vector
operations for all line shape models (Lorentzian/Gaussian/Voigt)
inside the processing loops. No hidden python loops or list
comprehensions inside the data updater.
3. Separation of Concerns: Maintain a clean architecture. Keep the
core physics calculators completely decoupled from the PyQt6
graphical thread loops, storing data indices inside structured
dictionaries.
4. Robust Parsing: Ensure the regex parsing engine handles ORCA
file structural variations by targeting boundaries dynamically from
the "IR SPECTRUM" string down to the terminal line of asterisks
("\n*").

Please provide a highly robust, modular, object-oriented production
script implementation of Phase 1, Phase 2, and Phase 3 to establish
a working multi-gas unit-switching dashboard pipeline. Include a
mock data array generator module for the ORCA parser output so the
system can be fully tested without a live local filesystem track.
Include clean docstrings and helpful annotations.
```